

HTML SVG

A Practical, From-Zero Learning Guide

Part 3 — Groups & Transformations

In Part 2 you learned how SVG paths describe geometry. Now you will learn how to organize that geometry and transform it without rewriting every coordinate.

Core idea: SVG transformations change the coordinate system or position of an element. Once you understand transformations, complex SVG illustrations become much easier to build.

1. Why Transformations Matter

Imagine you created a house from 20 shapes. If you want a second house 300 units to the right, you could manually change every x-coordinate — but that is inefficient. Transformations let you move the whole drawing as a unit.

```
<g transform="translate(300 0)">
  <!-- all house shapes -->
</g>
```

The same principle works for icons, characters, diagrams, charts, maps and UI graphics.

2. The Element

<g> means **group**. It lets you treat multiple SVG elements as one logical object.

```
<svg viewBox="0 0 400 250">

  <g>
    <rect x="50" y="100" width="120" height="90" fill="skyblue"/>
    <polygon points="40,100 110,40 180,100" fill="orange"/>
  </g>

</svg>
```

A group can contain shapes, paths, text, other groups and almost any SVG graphical element.

3. Styling a Group

Many visual properties can be inherited by children of a group.

```
<g fill="tomato" stroke="black" stroke-width="3">
  <circle cx="60" cy="60" r="30"/>
  <rect x="110" y="30" width="60" height="60"/>
  <polygon points="220,90 260,30 300,90"/>
</g>
```

Instead of repeating the same style on every element, you can put it on the group.

4. Group IDs

```
<g id="car">
  <rect x="50" y="100" width="200" height="70"/>
  <circle cx="90" cy="180" r="25"/>
  <circle cx="210" cy="180" r="25"/>
</g>
```

IDs are useful for CSS, JavaScript, accessibility and reusable SVG structures.

5. translate()

translate(x y) moves the coordinate system. The first value moves horizontally and the second moves vertically.

```
<rect x="20" y="30"
      width="80" height="50"
      fill="tomato"
      transform="translate(100 40)" />
```

The original rectangle is still defined at x=20, y=30, but its transformed position is shifted.

6. Translating a Group

```
<g transform="translate(200 50)">
  <rect x="0" y="0" width="100" height="60" fill="skyblue"/>
</g>
```

```
<circle cx="50" cy="90" r="25" fill="orange"/>
</g>
```

Every child in the group is affected by the group's transformation.

7. Negative Translation

```
<g transform="translate(-50 -20)">
  <circle cx="200" cy="150" r="40"/>
</g>
```

Negative values move the drawing left and/or upward.

8. scale()

scale() changes the size of an element or group.

```
<rect x="50" y="50"
      width="100" height="60"
      fill="tomato"
      transform="scale(2)" />
```

scale(2) doubles the coordinate system in both x and y directions.

9. Non-uniform Scaling

```
<rect x="50" y="50"
      width="100" height="60"
      fill="tomato"
      transform="scale(2 0.5)" />
```

scale(x y) can scale horizontal and vertical dimensions differently.

10. Negative Scale — Reflection

```
<g transform="scale(-1 1)">
  <rect x="-200" y="50"
        width="100" height="80"
        fill="skyblue"/>
</g>
```

A negative scale can mirror geometry. However, because the coordinate system also flips, you often combine it with a translation to place the reflected object where you want it.

11. rotate()

The basic form is **rotate(angle)**. Angles are measured in degrees.

```
<rect x="120" y="70"
      width="100" height="60"
      fill="orange"
      transform="rotate(30)" />
```

Positive rotation is clockwise in the SVG coordinate system because the y-axis points downward.

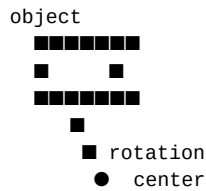
12. Rotating Around a Specific Point

The more complete form is **rotate(angle cx cy)**.

```
<rect x="140" y="80"
      width="80" height="50"
      fill="skyblue"
      transform="rotate(45 180 105)" />
```

Here the rectangle rotates 45 degrees around the point (180,105), rather than around the origin.

13. Understanding the Rotation Center



When no center is supplied, the transform operates around the current coordinate system's origin. Specifying `cx` and `cy` gives you precise control.

14. `skewX()`

Horizontal skew slants the shape by changing `x` according to `y`.

```
<rect x="60" y="60"
      width="100" height="70"
      fill="tomato"
      transform="skewX(25)" />
```

Skew is useful for italic-like effects, perspective-style illustrations and geometric designs.

15. `skewY()`

```
<rect x="60" y="60"
      width="100" height="70"
      fill="skyblue"
      transform="skewY(25)" />
```

`skewY()` slants the geometry in the vertical direction.

16. Transform Shorthand

SVG also supports the **`matrix()`** transform. It represents translation, scaling, rotation and skewing mathematically in one transformation matrix.

```
transform="matrix(a b c d e f)"
```

You do not need to memorize matrix values at this stage. Design software often generates matrix transforms automatically. Understanding the simpler transform functions first is much more important.

17. Combining Transformations

```
<rect x="50" y="50"
      width="80" height="50"
      fill="orange"
      transform="translate(100 40) rotate(30) scale(1.2)" />
```

Multiple transformations can be written in one transform attribute.

Important: transformation order matters. Changing the order can produce a completely different result.

18. Why Transform Order Matters

```
transform="translate(100 0) rotate(45)"
transform="rotate(45) translate(100 0)"
```

These are not generally equivalent. One transformation changes the coordinate system used by the next transformation. When debugging an SVG, test transformations one at a time.

19. Nested Groups

Groups can be placed inside other groups. This creates a hierarchy of coordinate systems.

```
<g transform="translate(100 50)">
  <g transform="rotate(20)">
    <g transform="scale(1.5)">
      <circle cx="50" cy="50" r="30"/>
    </g>
  </g>
</g>
```

This is extremely useful for complex illustrations. Think of the outer group as a body, an inner group as an arm, and another inner group as a hand.

20. A Practical Example — Robot

```
<svg viewBox="0 0 500 400">

  <g id="robot" transform="translate(100 50)">

    <!-- Head -->
    <rect x="100" y="30" width="160" height="100"
      rx="20" fill="#cbd5e1"
      stroke="#111827" stroke-width="5"/>

    <circle cx="145" cy="80" r="15" fill="#22c55e"/>
    <circle cx="215" cy="80" r="15" fill="#22c55e"/>

    <!-- Body -->
    <rect x="80" y="150" width="200" height="120"
      rx="15" fill="#94a3b8"
      stroke="#111827" stroke-width="5"/>

    <!-- Arm group -->
    <g transform="rotate(-15 80 170)">
      <rect x="35" y="150" width="45" height="120"
        rx="20" fill="#64748b"/>
    </g>

    <g transform="rotate(15 280 170)">
      <rect x="280" y="150" width="45" height="120"
        rx="20" fill="#64748b"/>
    </g>

  </g>

</svg>
```

The robot demonstrates grouping, nesting and rotation around selected points. Try changing the outer translate and the arm rotation.

21. transform-origin in CSS

When SVG is styled through CSS, **transform-origin** can be useful for controlling the point around which CSS transforms operate.

```
<style>
  .wheel {
    transform-origin: center;
    transform-box: fill-box;
    transition: transform 0.5s;
  }

  .wheel:hover {
    transform: rotate(180deg);
  }
</style>
```

```

<svg viewBox="0 0 200 200">
  <circle class="wheel"
    cx="100" cy="100" r="60"
    fill="none"
    stroke="black"
    stroke-width="8"/>
</svg>

```

SVG CSS transform behavior can differ from ordinary HTML boxes, so **transform-box** is worth learning when building interactive SVG.

22. Transform vs Changing Coordinates

Suppose you have a circle at cx=50. You could move it by changing cx to 150, or you could use `translate(100 0)`.

```

<!-- Coordinate change -->
<circle cx="150" cy="80" r="30"/>

<!-- Transformation -->
<circle cx="50" cy="80" r="30"
  transform="translate(100 0)"/>

```

Both can produce the same visible position, but transformations are much more powerful when you need to move an entire group.

23. Reusing a Group with

The **<use>** element lets you reuse an existing SVG definition.

```

<svg viewBox="0 0 500 200">

  <defs>
    <g id="star">
      <polygon points="50,5 61,35 95,35
        68,55 78,90 50,70
        22,90 32,55 5,35
        39,35"
        fill="gold"/>
    </g>
  </defs>

  <use href="#star" x="20" y="20"/>
  <use href="#star" x="140" y="20"/>
  <use href="#star" x="260" y="20"/>

</svg>

```

This is an introduction to reusable SVG. Part 7 will explore **<defs>**, **<symbol>**, sprites and reusable components in much greater depth.

24. Coordinate Systems — The Big Picture

Every SVG starts with a coordinate system. Transformations effectively create new coordinate systems for elements and groups.

- The root SVG has its own coordinate system.
- A group can transform that coordinate system.
- Children draw using the transformed coordinate system.
- Nested groups can create additional transformed coordinate systems.
- This hierarchy makes complex drawings manageable.

25. Common Transformation Mistakes

- Forgetting that SVG's y-axis points downward.
- Assuming transformations always happen in the order you mentally imagine.
- Rotating around the wrong center.
- Using scale without considering the coordinate origin.
- Trying to manually edit hundreds of coordinates instead of grouping elements.
- Applying transforms to individual shapes when the entire group should move.
- Forgetting that nested transforms accumulate.

26. Mini Project — Repeated Houses

```
<svg viewBox="0 0 700 300">

  <defs>
    <g id="house">
      <rect x="0" y="70"
        width="140" height="100"
        fill="#60a5fa"/>

      <path d="M -15 70 L 70 5 L 155 70 Z"
        fill="#f59e0b"/>

      <rect x="55" y="115"
        width="30" height="55"
        fill="#92400e"/>
    </g>
  </defs>

  <use href="#house" x="30" y="70"/>
  <use href="#house" x="250" y="70"
    transform="scale(0.8)"/>
  <use href="#house" x="470" y="70"
    transform="scale(1.2)"/>

</svg>
```

Experiment with translation, rotation and scale. Try creating a row of houses with different sizes and angles.

27. Exercises

- Create a group containing a circle, rectangle and path. Move the whole group with `translate()`.
- Rotate a square 45 degrees around its center.
- Create two identical shapes and mirror one using `scale(-1 1)`.
- Create a simple character and rotate only its arms.
- Create a tree group and reuse it five times with different translations.
- Create a car using nested groups and rotate its wheels.
- Experiment with two transform strings containing the same functions in different orders.
- Recreate one of your Part 2 projects using groups and transformations.

28. Quick Reference

Feature	Purpose	Example
<g>	Group elements	<g>...</g>
translate	Move	translate(50 20)
scale	Resize	scale(2)
rotate	Rotate	rotate(45)
rotate with center	Rotate around point	rotate(45 100 100)
skewX	Horizontal slant	skewX(20)
skewY	Vertical slant	skewY(20)
matrix	General transform	matrix(a b c d e f)
<use>	Reuse an element	use href="#id"

29. What's Next?

Part 4 will focus on **SVG colors and advanced painting**: linear gradients, radial gradients, gradient stops, gradient coordinate systems, patterns, opacity, `currentColor`, CSS variables and advanced stroke techniques.